PLAN INTEGRAL DE PRUEBAS

Caja blanca, caja negra y seguridad para FusaShop

Versión 1.0 · 23 de octubre de 2026
Proyecto: plataforma de comercio electrónico FusaShop
Base técnica revisada: Laravel 12, PHP 8.3+, PHPUnit 11
Preparado para ejecución en ambiente local y staging autorizado

Propósito Definir qué validar, cómo hacerlo, con qué herramientas y qué evidencias conservar antes de publicar cambios de FusaShop. Este documento es un plan; no afirma que las pruebas ni los escaneos se hayan ejecutado.

Resumen ejecutivo

FusaShop ya cuenta con pruebas funcionales de Laravel que cubren partes relevantes de registro y verificación por correo, checkout, Wompi, inventario, webhooks, facturación, roles administrativos, reportes, promociones y FusaBot. El repositorio configura PHPUnit con bases de datos SQLite en memoria y tiene suites Unit y Feature. El siguiente paso recomendado es ampliar pruebas de lógica y límites, validar la aplicación desplegada desde fuera con Postman y ZAP, y convertir los controles repetibles en un flujo de integración continua.

La prioridad del plan es proteger dinero, inventario, datos personales y permisos. Para cada versión se ejecutan primero pruebas automatizadas críticas, después pruebas API y de interfaz, y finalmente análisis de seguridad y rendimiento en un ambiente separado de producción.

1. Identificación y contexto

Elemento	Definición
Producto	FusaShop, tienda electrónica con consumidores, comercios y personal analista/administrador.
Stack observado	Laravel ^12.0, PHP ^8.3, PHPUnit ^11.4; Sanctum; base de pruebas SQLite :memory:; colas sync en pruebas.
Integraciones identificadas	Wompi, Envia, Factus, correo y funciones de asistente/MCP.
Repositorio revisado	composer.json, phpunit.xml, rutas API y web, app/Services y nombres/métodos de tests/Feature.
Fecha de corte	23 de octubre de 2026. Revisar versiones y configuración cuando cambie el despliegue.

1.1 Objetivo

Encontrar defectos funcionales, de autorización, integración, datos, rendimiento y seguridad; demostrar con evidencia reproducible que los flujos prioritarios satisfacen los resultados esperados; y establecer criterios de salida y seguimiento de hallazgos.

1.2 Alcance

- Registro, inicio de sesión, verificación OTP de seis dígitos, reenvío de código, recuperación de contraseña y sesiones.
- Catálogo, productos, perfiles de comercio, promociones, carrito, FusaCoins, checkout selectivo, pedidos y cancelaciones.
- Reserva, confirmación y liberación de inventario; cobro Wompi y eventos de webhook.
- Cotización y seguimiento de Envia, facturación Factus, notificaciones y reintentos de integraciones.
- Roles consumidor, comerciante, analista y administrador; propiedad de pedidos, exportaciones y archivos.
- Rutas API con Sanctum, webhooks y MCP; FusaBot; paneles, reportes y auditoría.
- Comportamiento de la web en móvil/escritorio, accesibilidad básica y rendimiento de flujos críticos.

1.3 Fuera de alcance y límites

No se certifica DIAN, Wompi, Envia ni Factus; el plan comprueba la integración de FusaShop con ambientes de prueba y las respuestas observables. No cubre infraestructura Hostinger ni servicios de terceros más allá de la configuración y disponibilidad que el equipo pueda controlar. Las pruebas activas de ZAP y carga deben ejecutarse exclusivamente contra un entorno autorizado con datos sintéticos.

2. Estrategia y niveles de prueba

Nivel	Enfoque	Técnicas/herramientas	Cuándo
Estático	Errores de estilo, dependencias vulnerables, tipos y patrones inseguros.	Laravel Pint; Composer audit; npm audit si aplica; Larastan/PHPStan como incorporación propuesta.	Cada PR / semanal.
Caja blanca	Lógica interna, ramas, excepciones, transacciones, estados, eventos y límites.	PHPUnit 11 + Laravel TestCase; factories; RefreshDatabase; fakes/mocks; cobertura con Xdebug o PCOV.	Cada PR.
Integración	Persistencia, colas, correo, archivos, APIs externas y webhooks.	Feature tests de Laravel; Http::fake; Queue::fake; Event::fake; Storage::fake; sandbox.	Cada PR + pre-release.
Caja negra	Contrato y comportamiento sin depender de implementación interna.	Postman Collections/CLI, navegador y checklist de historias.	Cada release candidata.
Seguridad DAST	Configuración HTTP, entradas, autenticación y exposición de vulnerabilidades.	OWASP ZAP Baseline; ZAP Automation Framework/API scan con alcance restringido.	Staging por release y mensual.
Rendimiento	Latencia, errores, concurrencia y recuperación ante picos.	k6 o JMeter (propuesta); métricas del servidor y base de datos.	Antes de campañas y cambios de checkout.
Compatibilidad UX	Diseño adaptable, navegación, validación, errores y accesibilidad básica.	Playwright o Laravel Dusk (si se incorpora); navegadores reales/emulación móvil.	Smoke por release.

3. Herramientas y uso

3.1 Laravel y PHPUnit para caja blanca

Usar las suites que ya están configuradas en phpunit.xml. Los Feature tests son la primera línea para rutas y flujos HTTP; agregar Unit tests a reglas puras de servicios y cálculos. Laravel ofrece RefreshDatabase, factories y aserciones de base de datos. Para no depender de servicios externos en pruebas, simular HTTP, colas, eventos, correo, notificaciones y almacenamiento. Verificar cobertura con Xdebug o PCOV y revisar por módulo: el porcentaje no sustituye los casos de negocio.

Comando/acción sugerida	Uso
php artisan test	Ejecutar las suites de Laravel; usar --testsuite=Unit o --testsuite=Feature para aislar.
php artisan test --filter=CheckoutFlowTest	Ejecutar la suite crítica de checkout durante depuración.
vendor/bin/phpunit --coverage-html reports/coverage	Generar cobertura si Xdebug/PCOV está instalado y habilitado.
vendor/bin/pint --test	Detectar problemas de formato Laravel, si Pint se incorpora al proyecto.
composer audit	Revisar avisos de seguridad de dependencias Composer.

Estado observado composer.json declara PHPUnit, pero no configura scripts de test, Pint, Larastan ni la generación de cobertura. Se presentan como pasos recomendados; verificar instalación antes de automatizarlos.

3.2 Postman para caja negra de API

– Crear colección “FusaShop API” con carpetas Auth, Products, Cart, Orders, Analytics, Webhooks y MCP. Importar/registrar las rutas API desde routes/api.php y mantener una matriz de las rutas web relevantes.

– Crear entornos Local y Staging con baseUrl, tokens de consumidor/comercio/analista, identificadores de datos sintéticos y secretos de sandbox en variables secretas; no guardar claves reales en el repositorio ni exportaciones compartidas.

– Para cada solicitud validar código HTTP, Content-Type, estructura JSON, campos obligatorios, invariantes comerciales y tiempos. Usar pre-request/post-response scripts para encadenar login → carrito → pedido y conservar IDs de pruebas.

– Agregar pruebas negativas: sin token, token inválido, rol equivocado, recurso de otro usuario, campos ausentes/mal tipados, límites de cantidad, parámetros inesperados, duplicados y rate limits.

– Ejecutar manualmente en Collection Runner y después con Postman CLI en pipeline. Guardar reporte de ejecución y vincularlo al commit/release. Preparar datos y limpieza para que la colección pueda repetirse sin efectos colaterales.

Ejemplo de aserción post-response para una respuesta JSON:

Script de ejemplo

pm.test("responde correctamente", () => { pm.response.to.have.status(200); });
pm.test("es JSON", () => { pm.response.to.be.json; });
const body = pm.response.json();
pm.test("incluye datos", () => { pm.expect(body).to.have.property("data"); });

3.3 OWASP ZAP

ZAP ayuda a descubrir problemas en la superficie HTTP visible. El Baseline Scan rastrea el sitio y espera el análisis pasivo; reporta hallazgos sin lanzar ataques activos. Para una revisión más profunda, la Automation Framework permite automatizar alcance, spider, importación OpenAPI/Postman, espera del escaneo pasivo, análisis activo y reporte. Un escaneo activo sí envía cargas de prueba, por lo que se agenda solo en staging aislado y autorizado.

– Confirmar URL exacta, ventana, responsables y alcance; usar staging, datos ficticios y cuentas de prueba por rol.

– Ejecutar baseline pasivo contra el sitio y revisar alertas, URLs de ejemplo y falsos positivos.

– Si existe OpenAPI, importarla; si no, importar una colección Postman o recorrer las páginas. Definir contexto autenticado para áreas privadas y excluir pagos reales, logout destructivo, eliminación y endpoints de terceros.

– Ejecutar análisis activo limitado solo después de aprobarlo dentro del equipo y comprobar respaldo/restauración del staging. Limitar tasa/concurrencia y excluir sistemas externos.

– Clasificar cada alerta por severidad y evidencia; reproducir manualmente, asignar responsable y volver a escanear tras corregir. Adjuntar HTML/JSON y versión de ZAP al expediente de release.

Regla operativa No ejecutar análisis activo, pruebas de carga ni repetición de webhooks contra producción. No introducir credenciales de producción en ZAP o Postman.

3.4 Otras herramientas recomendadas

Herramienta	Aporte y decisión
Xdebug o PCOV	Cobertura de líneas/ramas para PHPUnit. Elegir uno compatible con PHP del runner; no ejecutar en producción.
Laravel Pint	Formato automático de código; hoy no aparece como dependencia declarada.
Larastan/PHPStan	Análisis estático de tipos y errores; incorporar gradualmente con nivel inicial acordado.
Playwright o Laravel Dusk	Validación de navegador y regresión responsive; elegir uno, evitar doble mantenimiento.
k6 o JMeter	Carga controlada de rutas públicas/API con cuentas y datos sintéticos; acordar umbrales antes de medir.
CI (GitHub Actions u otro)	Automatizar PHPUnit, análisis de dependencias, lint, Postman CLI y generación de reportes. Confirmar plataforma real del repositorio.

4. Modelo de ambiente y datos

Elemento	Configuración propuesta
Local/CI	Base aislada temporal; phpunit.xml usa SQLite en memoria, cola sync y correo array. No compartir DB real.
Staging	Despliegue semejante a Hostinger, separado de producción; configuración de sandbox para Wompi/Envia/Factus cuando esté disponible.
Datos	Usuarios y direcciones sintéticos; códigos postales válidos e inválidos para Colombia; productos de prueba con stock controlado.
Servicios externos	Simular respuestas en pruebas automatizadas. En staging, usar sandbox o mocks controlados; definir timeout, caída, 401, 429, 5xx y respuestas malformadas.
Trazabilidad	Identificador por ejecución, versión/commit, ambiente, fecha/hora, navegador, colección y reporte ZAP.
Limpieza	Restaurar base/fixtures; invalidar tokens; borrar órdenes de prueba y archivos temporales sin tocar datos de usuarios reales.

Antes de probar concurrencia o migraciones, verificar que las diferencias entre SQLite y MariaDB/MySQL de producción no cambien el resultado: restricciones, tipos, collation, transacciones, bloqueos y consultas. Mantener pruebas rápidas en SQLite y programar una etapa de integración con el motor real en staging.

5. Casos de prueba priorizados

Las filas describen casos propuestos para ejecución. “Cobertura existente” significa que hay pruebas con un propósito similar en el repositorio, no que se garantice la ausencia de defectos. P0 bloquea publicación; P1 es alta prioridad; P2 mejora regresión.

ID	Caso	Pasos resumidos / resultado esperado	Prioridad · herramienta	Situación en repo
AUTH-01	Registro + OTP válido	Crear usuario, recibir código de 6 dígitos y verificar. Confirmar cuenta y sesión/redirect correctos.	P1 · PHPUnit/Postman	Existe suite de registro/OTP
AUTH-02	OTP incorrecto, vencido, reutilizado	Probar código erróneo, expiración y segundo uso. Debe rechazar sin verificar cuenta.	P1 · PHPUnit/Postman	Existe incorrecto/correcto; ampliar expiración/reuso
AUTH-03	Abuso de OTP/login	Repetir solicitudes hasta el límite; validar 429/espera y que no se filtra el código.	P1 · Laravel/Postman/ZAP	Throttles observados en web; verificar API
AUTH-04	Acceso por roles y sesión	Consumidor y comercio intentan rutas de analista; token inválido/ausente; sesión revocada. Esperar 401/403 y ningún dato sensible.	P0 · PHPUnit/Postman/ZAP	Existe cobertura admin parcial
MER-01	Aislamiento entre comercios	Comercio A lee/edita pedido o producto de B por ID directo. Debe denegar sin cambiar datos.	P0 · PHPUnit/Postman	Ampliar IDOR/API
CAT-01	Validación de producto y carga	Nombre/descr. con HTML, precio límite, imagen tipo/tamaño no permitido. Rechazo seguro y salida escapada.	P1 · PHPUnit/ZAP	Añadir pruebas negativas
CART-01	Manipulación de precio	Alterar precio/total enviado desde cliente. El backend recalcula con precio vigente o rechaza.	P0 · PHPUnit/Postman	Probar explícitamente
STK-01	Stock insuficiente y concurrencia	Dos compras simultáneas compiten por última unidad. Solo una confirma; inventario no negativo.	P0 · PHPUnit + MariaDB	Existe stock insuficiente/reserva; sumar carrera real
STK-02	Vencimiento y cancelación reserva	Dejar pedido pendiente hasta expiración, cancelar permitido y reintentar. Stock y FusaCoins se restauran exactamente una vez.	P0 · PHPUnit	Existe expiración/cancelación
CHK-01	Checkout selectivo y descuentos	Comprar subset del carrito con descuento/FusaCoins. Solo se cobra y retira del carrito lo seleccionado; valores son consistentes.	P0 · PHPUnit/Postman	Existe cobertura de checkout selectivo
PAY-01	Webhook Wompi firma y monto	Firma inválida, timestamp vencido y monto distinto no aprueban pedido ni disparan efectos.	P0 · PHPUnit/Postman	Existe firma/timestamp/monto
PAY-02	Webhook duplicado/fuera de orden	Reenviar evento aprobado o enviar rechazo tardío tras aprobación. Estado/stock/factura no se duplican ni retroceden.	P0 · PHPUnit/Postman	Existe idempotencia; ampliar orden de eventos
PAY-03	Proveedor caído/reintento	Simular timeout, 401, 429, 5xx y JSON inválido. Registrar error, mantener pedido íntegro y permitir reintento seguro.	P0 · PHPUnit/staging	Existe fallo externo general
SHP-01	Cotizar Envia con destinos	Probar código postal válido, inválido, vacío y ciudad no cubierta; dirección completa e incompleta. Respuesta clara y sin error silencioso.	P0 · PHPUnit/Postman	Crear pruebas de frontera; sandbox
SHP-02	401 credencial Envia	Forzar token ausente/incorrecto y expirado. No revelar secreto; registrar 401; mostrar error recuperable.	P0 · PHPUnit/Postman	El incidente mostrado hace este caso crítico
SHP-03	Timeout y reintento de guía	Proveedor lento/caído; reintentar acción y comprobar que no se dupliquen guías ni cobros.	P1 · PHPUnit/staging	Confirmar política de idempotencia
INV-01	Factura Factus	Pago aprobado crea factura con totales/datos fiscales correctos y entrega archivo permitido.	P0 · PHPUnit/sandbox	Existe flujo parcial
INV-02	Reintento y factura duplicada	Factus 5xx y reintento posterior; una sola factura lógica por pedido y trazabilidad de errores.	P0 · PHPUnit/staging	Existe fallo genérico de API
API-01	Contrato API autenticada	Login, productos, carrito, pedidos, analytics; validar status, JSON, campos, errores y paginación.	P1 · Postman CLI	Cobertura API por consolidar
API-02	Sanctum y acceso a recursos	Token de consumidor consulta pedido propio y ajeno; solo propio; logout invalida credencial.	P0 · PHPUnit/Postman	Existe api routes; ampliar escenarios
BOT-01	FusaBot/MCP autorización y datos	Sin token, token incorrecto, payload malformado y consulta de pedido ajeno. Debe limitar datos y responder con error seguro.	P0 · PHPUnit/Postman/ZAP	Existe prueba MCP token/lookup
ADM-01	Cambios administrativos y auditoría	Cambiar estado/KYC/configuración; validar autorización, actor, marca temporal y valores auditados donde aplique.	P1 · PHPUnit	Existe governance; confirmar auditoría
WEB-01	XSS/CSRF/archivos/IDOR	Probar entradas reflejadas/almacenadas, formularios, enlaces privados y rutas de archivos con usuario adverso.	P0 · ZAP + revisión manual	No sustituye pruebas de autorización
UI-01	Responsive y contenido/renderizado	Móvil pequeño y escritorio; navegación/menú/OTP/checkout; revisar overflow, contenido que aparece tarde, errores consola y recarga.	P1 · Playwright/Dusk manual	No inferir cobertura por tests server
PERF-01	Carga lectura y checkout	Crecimiento gradual de solicitudes sintéticas; medir p95, tasa de error, CPU, DB, colas y degradación.	P1 · k6/JMeter	Definir umbrales con negocio
DEP-01	Dependencias y configuración	Auditar Composer/NPM, secretos/APP_DEBUG, HTTPS, cookies, headers, permisos y configuración del despliegue.	P0 · CI/ZAP/manual	Validar por cada release

6. Caja blanca: diseño y cobertura

6.1 Objetos y ramas críticas

- CheckoutService, StockService y OrderPaymentService: cálculos, límites, transacciones, commit/rollback, reserva y liberación.
- WompiService, EnviaService y FactusService: construcción de solicitudes, configuración, firma, parseo, timeout, errores y reintentos.
- Controllers/Requests/Policies: validación, autorización, propiedad del recurso y resultados HTTP.
- Jobs/listeners/eventos: ejecución solo tras condición correcta, reintentos idempotentes y manejo de fallos.
- OTP, descuentos, FusaCoins, comisiones y estados: frontera numérica/temporal, redondeos, doble aplicación y reglas de transición.

6.2 Matriz de cobertura propuesta

Indicador	Meta inicial propuesta	Aplicación
Pruebas de regresión P0	100 % pasan antes de publicar; cero fallo conocido sin decisión documentada.	Checkout, stock, pagos, roles, tarifas de envío, facturas.
Cobertura de ramas	≥80 % en servicios de pago, stock y checkout al incorporar coverage; revisar ramas críticas sin cubrir.	No usar cobertura global como único criterio.
Escenarios de autorización	Cada grupo de rol prueba permitido, denegado y acceso a ID ajeno en rutas protegidas.	Feature tests + colección Postman.
Integraciones externas	Éxito, autenticación fallida, timeout, rate limit, 5xx, respuesta inválida y repetición.	Fakes automatizados + sandbox.
Datos y persistencia	Afirmar estado antes/después en pedidos, inventario, créditos, logs y facturas.	assertDatabaseHas/Missing y pruebas transaccionales.

Las metas son una propuesta para aprobación del equipo y deben calibrarse con la cobertura base y el costo de mantener pruebas. Para lógica con impacto financiero o de permisos, se prioriza la cobertura de ramas y estados sobre un porcentaje general.

7. Caja negra: diseño Postman y ZAP

7.1 Estructura sugerida de la colección Postman

Carpeta	Solicitudes/casos mínimos
00 Salud y autenticación	GET health/landing si existe; login correcto/incorrecto; registro; OTP; logout; token faltante/expirado.
01 Catálogo	Listado, detalle, paginación, filtros, producto inexistente, límites/parámetros inválidos.
02 Carrito y checkout	Agregar, actualizar, borrar, sin stock, cambio de precio, checkout total/parcial, duplicados.
03 Pedidos	Listar propio, detalle propio/ajeno, cancelar antes/después de despacho, factura y tracking.
04 Comercio y analista	Productos/reportes por rol, datos ajenos, exportación, acción no autorizada.
05 Integraciones	Webhooks Wompi/Envia con firmas/secreto válidos e inválidos, repetidos y cuerpo malformado.
06 MCP y asistente	Token y payload correctos, esquema inválido, límite, datos ajenos y respuesta controlada.

7.2 Esquema de validaciones por respuesta

- HTTP: status esperado, Content-Type, headers de seguridad pertinentes, no redirects inesperados ni trazas de error.
- Contrato: objeto/campos obligatorios y tipos; JSON válido; mensajes de validación consistentes sin detalles internos.
- Negocio: total recalculado, existencia de IDs, transición de estado permitida, inventario y FusaCoins coherentes.
- Seguridad: respuesta 401/403 para falta de identidad/permisos; no exponer datos personales, tokens, secretos, stack traces o datos de otro tenant/usuario.
- Rendimiento: umbral por endpoint acordado y reportado, no usar una sola medición como certificación de capacidad.

7.3 Perfil de revisión ZAP

Ejecución	Cobertura	Artefacto
ZAP Baseline pasivo	Rastrear páginas públicas y revisar headers, cookies, contenido y alertas pasivas.	Reporte HTML/JSON, versión y URL/ambiente.
ZAP con sesión	Contexto autenticado separado para consumidor, comercio y analista; inspección de áreas privadas y controles de acceso.	Contexto redactado, cuentas sintéticas, hallazgos por rol.
API Scan / Automation Framework	Importar OpenAPI si está disponible o colección Postman; validar alcance y escanear staging.	Plan YAML, log, alerts y reporte reproducible.
Re-test	Repetir tras parche y confirmar alerta desaparecida, sin regresión funcional.	Comparativo antes/después y ticket cerrado con evidencia.

ZAP es una herramienta DAST automatizada: no demuestra que el sistema esté libre de vulnerabilidades. Complementar con revisión de código, pruebas de permisos y análisis manual de las alertas relevantes.

8. Ejecución, orden y calendario

Fase	Actividad	Salida / responsable sugerido
0. Preparación	Acordar alcance, responsable, versión, staging, datos, cuentas, credenciales sandbox, respaldo y ventana.	Ficha de ejecución · líder QA + desarrollo/operaciones.
1. PR/CI	Formato y dependencias, PHPUnit Unit/Feature, cobertura enfocada y revisión de cambios.	Pipeline verde; desarrollador autor.
2. Integración	Pruebas con MariaDB/MySQL equivalente, colas, jobs, correo fake y mocks/sandbox.	Log y base limpia · QA/desarrollo.
3. API caja negra	Correr colección Postman en staging, validar roles, casos positivos y negativos.	Reporte CLI con commit y ambiente · QA.
4. UI/regresión	Flujos principales en móvil y escritorio; responsive; consola y accesibilidad básica.	Checklist y capturas · QA/producto.
5. Seguridad	ZAP baseline; revisión de alertas; activo limitado y autorizado en staging; dependencias.	Reporte ZAP y tickets priorizados · seguridad/operaciones.
6. Rendimiento	Carga gradual acordada; observar p95, errores, DB y colas; recuperación.	Curva y resumen con umbrales · QA/operaciones.
7. Cierre	Resolver bloqueos, ejecutar smoke final, documentar riesgos aceptados y archivar evidencia.	Acta de salida de release · responsable del producto.

Cadencia recomendada: Unit/Feature en cada PR; Postman smoke y ZAP pasivo en cada versión candidata; ZAP autenticado/activo y rendimiento antes de releases de riesgo, campañas y cambios en pagos/inventario. Ajustar frecuencia a infraestructura y volumen de cambios.

9. Criterios de aceptación y severidad

9.1 Entrada

- Versión y commit identificados; despliegue de staging accesible y configuración de prueba confirmada.
- Datos sintéticos cargados; servicios externos dirigidos a sandbox/mock; alertas y logs consultables.
- Colecciones, scripts, cuentas por rol y alcance ZAP revisados. Ventana y contacto de interrupción definidos.

9.2 Salida / decisión de publicación

- Todos los casos P0 ejecutados y aprobados; los P1 fallidos tienen ticket, responsable y decisión explícita.
- Cero defectos críticos abiertos en pagos, exposición de datos, autorización o corrupción de stock.
- No quedan alertas ZAP críticas/altas sin corregir o con aceptación de riesgo escrita y fecha de vencimiento.
- Errores del pipeline y dependencias resueltos o justificados por responsable; reportes asociados al commit.
- Ambiente restaurado y operaciones de prueba conciliadas; evidencia y limitaciones guardadas.

9.3 Severidad de defectos

Nivel	Ejemplo	Tratamiento
Crítica / P0	Cobro incorrecto, fuga de datos, bypass de permisos, fraude o sobreventa con impacto real.	Detener release; triage inmediato y re-test obligatorio.
Alta / P1	Flujo esencial no disponible, integración falla de forma sostenida, factura incorrecta sin alternativa.	Corregir antes de release o aceptación formal de riesgo.
Media / P2	Función secundaria degradada con alternativa segura; error limitado en datos no sensibles.	Plan de corrección priorizado; validar impacto con producto.
Baja / P3	Defecto visual o mensaje menor sin impacto funcional/seguridad.	Registrar y planear en backlog.

10. Registro de defecto y evidencias

Campo	Qué registrar
Identificación	ID, título conciso, fecha, persona reportante, ambiente, commit/build.
Reproducción	Precondición, rol, datos sintéticos, pasos numerados y frecuencia.
Resultado	Esperado frente a obtenido; status/response, capturas o log redactado.
Impacto	Módulo, usuarios afectados, riesgo de dinero/datos/operación; severidad y prioridad.
Evidencia técnica	Request/response saneados, hora/ID de correlación, log relevante, reporte PHPUnit/Postman/ZAP.
Cierre	Commit correctivo, re-test, prueba de regresión relacionada y quien acepta/cierra.

Privacidad de evidencia Redactar tokens, claves, correos personales, números de documento y direcciones antes de adjuntar capturas, colecciones o reportes. Mantener el acceso limitado al equipo del proyecto.

11. Trazabilidad resumida

Área FusaShop	Prueba Laravel	Postman	ZAP	Carga/UI
Auth, OTP y roles	Feature + ramas	Auth/no auth/roles	Sesión, cookies, formularios	OTP UI y rate limit
Catálogo/carrito/checkout	Unit + Feature + DB	Contrato + reglas	Entradas/IDOR	picos de catálogo y compra móvil
Stock/pagos	Transacciones + webhook	firma/duplicados/estados	API autenticada	checkout concurrente
Envia/Factus	HTTP fake + sandbox	errores y reintentos	API/rutas expuestas	timeout/degradación controlada
Reportes/admin/MCP	Policies/jobs/servicios	roles/payload/schema	alcance por sesión	render escritorio/móvil

12. Riesgos y decisiones pendientes

- Confirmar si staging usa exactamente el motor SQL y las versiones del hosting; phpunit.xml fuerza SQLite en memoria.
- Confirmar si existe definición OpenAPI; routes/api.php ofrece rutas API identificables, pero no se observó contrato OpenAPI en la revisión de archivos listados.
- Acordar el uso de ambientes sandbox de Wompi, Envia y Factus y las credenciales no productivas.
- Definir las reglas reales de expiración/idempotencia de eventos, retención de logs y auditoría, y los umbrales de rendimiento con producto.
- Verificar si FusaShop publica API para terceros y qué endpoints exactos están soportados antes de cerrar la colección Postman.
- Confirmar si se incorpora Pint, Larastan, cobertura y un runner CI; no aparecen declarados en composer.json en la revisión actual.

Referencias

Laravel. (s. f.). Database testing. Laravel 12.x. https://laravel.com/docs/12.x/database-testing

Laravel. (s. f.). HTTP tests. Laravel 12.x. https://laravel.com/docs/12.x/http-tests

Laravel. (s. f.). Testing. Laravel 12.x. https://laravel.com/docs/12.x/testing

OWASP Foundation. (s. f.). ZAP baseline scan. https://www.zaproxy.org/docs/docker/baseline-scan/

OWASP Foundation. (s. f.). Automation framework. ZAP. https://www.zaproxy.org/docs/automate/automation-framework/

OWASP Foundation. (s. f.). ZAP Docker documentation. https://www.zaproxy.org/docs/docker/

Postman. (s. f.). Write scripts to test API response data in Postman. https://learning.postman.com/docs/tests-and-scripts/write-scripts/test-scripts/

Postman. (s. f.). Run a collection using the Postman CLI. https://learning.postman.com/docs/postman-cli/postman-cli-run-collection

Postman. (s. f.). Test your API using the Collection Runner. https://learning.postman.com/docs/tests-and-scripts/running-collections/intro-to-collection-runs/

PHPUnit. (s. f.). Code coverage analysis. PHPUnit 11. https://docs.phpunit.de/en/11.5/code-coverage.html

Nota de revisión: documentación técnica consultada para orientar el plan. Confirmar compatibilidad de cada herramienta con el runtime y las políticas del proyecto antes de instalar o añadirla al pipeline.